

OMIS 105: Database Management Systems

Database Management Systems

Week 1 — Foundations

Instructor: Dr. Parsian

Course Overview

- **Course:** OMIS 105 — Database Management Systems
- **Prerequisite:** OMIS 30 (Intro to Programming)
- **Duration:** 10 weeks, 2 sessions × 2 hours/week
- **Tools:** DuckDB, SQL, Jupyter Notebooks
- **Domain:** E-commerce database (ShopSmart Inc.)

What You Will Learn

1. How databases store and organize data
2. The relational model and SQL
3. Database design and normalization
4. Performance tuning and indexing
5. Transactions and data integrity
6. Building a real-world database project

Week-by-Week Roadmap

Week	Topic
1	Foundations
2	Relational Thinking
3–5	SQL Mastery
6	Normalization
7	Performance
8	Transactions (ACID)
9	Capstone Project
10	Synthesis & Review

Session 1: Why Databases?

The Data Problem

Imagine you run **ShopSmart**, an online store:

- 64 products across 8 categories
- 40 customers placing orders daily
- Hundreds of orders with thousands of line items

How do you store and manage all this data?

Option 1: Flat Files (Spreadsheets)

```
product_id, name, category, price, stock  
1, Smartphone X12, Electronics, 299.99, 150  
2, Laptop Pro 15, Electronics, 899.99, 45  
...
```

Seems simple enough... right?

Problems with Flat Files

- **Redundancy:** Category "Electronics" repeated for every electronics product
- **Inconsistency:** What if someone types "Electronicss"?
- **No concurrent access:** Two employees editing the same file?
- **No security:** Everyone sees everything
- **Scale:** Try searching 10 million rows in Excel

Option 2: A Database

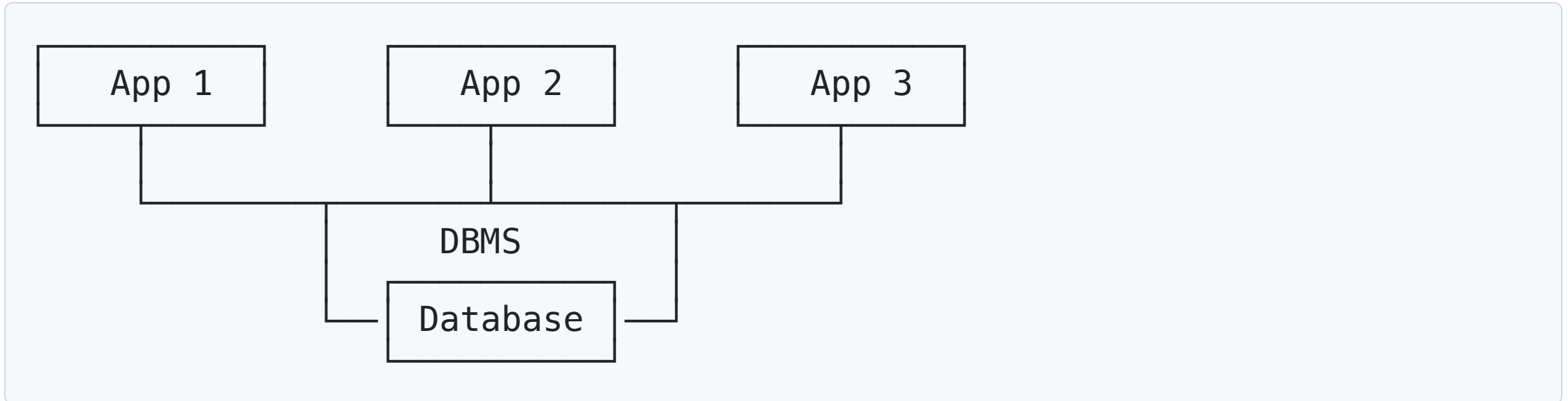
A **database** is an organized collection of structured data, stored electronically and managed by a **Database Management System (DBMS)**.

A DBMS provides:

- Structured storage
- Query language (SQL)
- Concurrent access
- Security and access control
- Data integrity enforcement

What Is a DBMS?

Database Management System — software that sits between applications and data.



Popular DBMS Software

DBMS	Type	Use Case
Oracle	Enterprise Relational	Banking, large corps
PostgreSQL	Open-source Relational	Web apps, analytics
MySQL	Open-source Relational	Web applications
SQL Server	Enterprise Relational	Microsoft ecosystem
MongoDB	Document (NoSQL)	Flexible schemas
DuckDB	Analytical Relational	Analytics, education

Why DuckDB for This Course?

- **Zero setup:** No server needed — runs in-process
- **Standard SQL:** Full SQL support
- **CSV-friendly:** Load data from CSV files directly
- **Fast:** Columnar storage, vectorized execution
- **Python integration:** Works great in Marimo/Jupyter notebooks
- **Free & open source**

Key Database Concepts

- Table
- Row
- Column
- Primary Key
- Foreign Key

Tables

A **table** is a collection of **related data** organized in rows and columns.

- Table name: products
- Column names:
 - product_id
 - product_name
 - category
 - price

product_id	product_name	category	price
1	Smartphone X12	Electronics	299.99
2	Laptop Pro 15	Electronics	899.99
3	Wireless Earbuds	Electronics	99.99

Rows and Columns

- **Row** (record/tuple): A single data entry
 - Example: All info about "Smartphone X12"
- **Column** (field/attribute): A single property
 - Example: All product prices
- **Cell**: The intersection of a row and column
 - Example: The price of Smartphone X12 = 299.99

Schema

A **schema** defines the structure of a database:

- What tables exist
- What columns each table has
- What data types each column holds
- What constraints apply

```
CREATE TABLE products (  
    product_id    INTEGER PRIMARY KEY,  
    product_name  VARCHAR,  
    category      VARCHAR,  
    price         DECIMAL(10,2),  
    stock_quantity INTEGER  
);
```

Data Types

Type	Description	Example
INTEGER	Whole numbers	42
DECIMAL(p,s)	Exact decimals	29.99
VARCHAR	Variable-length text	'Laptop Pro '
DATE	Calendar date	'2024-06-15 '
BOOLEAN	True/False	TRUE
TIMESTAMP	Date and time	'2024-06-15 14:30:00 '

Constraints

Rules that enforce data integrity:

Constraint	Purpose
PRIMARY KEY (PK)	Uniquely identifies each row
NOT NULL	Column cannot be empty
UNIQUE	No duplicate values allowed
CHECK	Custom validation rule
DEFAULT	Auto-fill value if none given
FOREIGN KEY (FK)	Links to another table (Week 2)

Example with Constraints

```
CREATE TABLE products (  
    product_id      INTEGER PRIMARY KEY,  
    product_name    VARCHAR NOT NULL,  
    category        VARCHAR NOT NULL,  
    price           DECIMAL(10,2)  
    CHECK (price > 0),  
    stock_quantity  INTEGER DEFAULT 0  
    CHECK (stock_quantity >= 0)  
);
```

Session 2: Hands-On with DuckDB

Installing DuckDB

Python (pip):

```
pip install duckdb
```

In a Marimo/Jupyter Notebook:

```
import duckdb

# Create an in-memory database
con = duckdb.connect()
print(con.sql("SELECT 'Hello, DuckDB!' AS greeting"))
```

Your First Query

```
import duckdb
con = duckdb.connect()

result = con.sql("SELECT 42 AS answer")
print(result)
```

Output:

answer int32
42

Loading CSV Data

```
import duckdb
con = duckdb.connect()

# Load products.csv directly
con.sql("""
    CREATE TABLE products AS
    SELECT * FROM read_csv_auto('./data/products.csv')
""")

# See what we loaded
con.sql("SELECT * FROM products LIMIT 5").show()
```

Examining Table Structure

```
-- Show all tables  
SHOW TABLES;  
  
-- Describe a table's columns  
DESCRIBE products;  
  
-- Count rows  
SELECT COUNT(*) AS total_products  
FROM products;
```

Basic SELECT

-- All columns, all rows

```
SELECT *  
FROM products;
```

-- Specific columns

```
SELECT product_name, price  
FROM products;
```

-- With a condition

```
SELECT product_name, price  
FROM products  
WHERE price > 100;
```

Anatomy of a SELECT Statement

```
SELECT    column1, column2    -- What to show
FROM      table_name         -- Where to look
WHERE     condition          -- Which rows
ORDER BY  column1            -- Sort results
LIMIT    10;                -- How many rows
```

Each clause has a purpose. We will master these in Weeks 3–5.

Filtering with WHERE

-- Exact match

```
SELECT * FROM products WHERE category = 'Electronics';
```

-- Numeric comparison

```
SELECT * FROM products WHERE price < 50;
```

-- Combining conditions

```
SELECT * FROM products  
WHERE category = 'Books' AND price < 30;
```

Sorting with ORDER BY

```
-- Ascending (default)
SELECT product_name, price
FROM products
ORDER BY price;

-- Descending
SELECT product_name, price
FROM products
ORDER BY price DESC;

-- Multiple columns
SELECT product_name, category, price
FROM products
ORDER BY category, price DESC;
```

Limiting Results

```
-- Top 5 most expensive products  
SELECT product_name, price  
FROM products  
ORDER BY price DESC  
LIMIT 5;
```

```
-- Skip first 10, then get 5  
SELECT product_name, price  
FROM products  
ORDER BY price DESC  
LIMIT 5 OFFSET 10;
```

Aliases

```
-- Column alias
SELECT product_name AS name,
       price AS unit_price
FROM products;

-- Computed column with alias
SELECT product_name,
       price * 1.0875 AS price_with_tax
FROM products;
```

DISTINCT Values

```
-- All unique categories  
SELECT DISTINCT category  
FROM products;
```

```
-- Count of unique categories  
SELECT COUNT(DISTINCT category) AS num_categories  
FROM products;
```

NULL Values

NULL means "unknown" or "missing" — not zero, not empty string.

```
-- Check for NULL
SELECT *
FROM products
WHERE stock_quantity IS NULL;

-- Check for NOT NULL
SELECT *
FROM products
WHERE stock_quantity IS NOT NULL;

-- CAUTION: X X X This does NOT work! X X X
-- SELECT * FROM products WHERE stock_quantity = NULL;
```

The LIKE Operator

Pattern matching for text:

```
-- Starts with 'S'  
SELECT * FROM products WHERE product_name LIKE 'S%';  
  
-- Contains 'Pro'  
SELECT * FROM products WHERE product_name LIKE '%Pro%';  
  
-- Exactly 3 characters  
SELECT * FROM products WHERE category LIKE '___';
```

% = any sequence of characters

_ = exactly one character

The IN Operator

```
-- Instead of multiple ORs  
SELECT * FROM products  
WHERE category IN ('Electronics', 'Books', 'Sports');
```

```
-- Equivalent but longer:  
SELECT * FROM products  
WHERE category = 'Electronics'  
       OR category = 'Books'  
       OR category = 'Sports';
```

BETWEEN Operator

```
-- Price range (inclusive)
SELECT product_name, price
FROM products
WHERE price BETWEEN 20 AND 100;

-- Equivalent to:
SELECT product_name, price
FROM products
WHERE price >= 20 AND price <= 100;
```

Basic Aggregate Functions

```
-- Count all products
SELECT COUNT(*) AS total FROM products;

-- Average price
SELECT AVG(price) AS avg_price FROM products;

-- Min and Max
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products;

-- Sum of stock
SELECT SUM(stock_quantity) AS total_stock
FROM products;
```

Combining Aggregates

```
SELECT
    COUNT(*) AS total_products,
    ROUND(AVG(price), 2) AS avg_price,
    MIN(price) AS min_price,
    MAX(price) AS max_price,
    SUM(stock_quantity) AS total_inventory
FROM products;
```

DuckDB Special Features

```
-- Read CSV without creating a table
SELECT * FROM read_csv_auto('./data/products.csv') LIMIT 5;

-- Export query results to CSV
COPY (SELECT * FROM products WHERE price > 100)
TO './data/expensive_products.csv' (HEADER, DELIMITER ',');
```

```
-- Get column statistics
SUMMARIZE products;
```

File System as Database

DuckDB can query files directly:

```
-- Query CSV file as if it were a table
SELECT category, COUNT(*) AS cnt
FROM read_csv_auto('./data/products.csv')
GROUP BY category
ORDER BY cnt DESC;
```

No `CREATE TABLE` needed!

Saving Your Work

```
# In-memory (default) – lost when you close  
con = duckdb.connect()  
  
# Persistent – saved to file  
con = duckdb.connect('shopsmart.duckdb')  
  
# Now all tables persist between sessions
```

Summary: Key Terms

Term	Definition
Database	Organized collection of structured data
DBMS	Software to manage databases
Table	Data organized in rows and columns
Row	Single record in a table
Column	Single attribute/field
Schema	Structure definition of a database
SQL	Language to interact with databases
Query	A request for data from a database

Summary: SQL So Far

```
SELECT columns FROM table  
WHERE conditions  
ORDER BY columns [ASC|DESC]  
LIMIT n;
```

Key operators: = , <> , < , > , LIKE , IN , BETWEEN , IS NULL

Aggregate functions: COUNT , SUM , AVG , MIN , MAX

What Is Next?

Week 2: Relational Thinking

- Primary and foreign keys
- Relationships between tables
- Entity-Relationship diagrams
- Multiple related tables

Practice Makes Perfect

- Complete **Lab 1** (loading data, basic queries)
- Explore the `./data/products.csv` dataset
- Try writing your own queries
- Install DuckDB and experiment!

Questions?

Thank you!